

DESIGN SPECIFICATION

Themis

System Architecture & Design Specification
SBOM and VEX security-intelligence platform — single Go binary backed by PostgreSQL

VERSION	v0.3.11 (active change: themis-ai-1 → v0.4.0)
DATE	22 July 2026
SOURCE	github.com/themis-project/themis @ commit 02f4c95
STATUS	Generated from repository sources and project documentation

Contents

1. Product Overview	4
What Themis Is	4
Operating Model	4
Capability Summary	4
Design Philosophy	5
Release Status	6
2. System Architecture	7
The Dependency Rule and the Four Code Layers	7
Dual Mechanical Enforcement	8
The Format-Leakage Rule	8
Package Responsibilities	9
Key Ports	10
Outside the Layers: internal/testutil	10
3. Runtime Design and API Surface	11
Process model and boot sequence	11
Dependency-injection composition root	12
Authentication and unauthenticated surfaces	12
Two endpoint families	12
API conventions	13
Error envelope	13
Command-line surface	14
4. Ingestion Pipeline and Background Processing	15
Transport-Agnostic Ingestion	15
Asynchronous by Default: the Job Queue	15
Pipeline Stages and Lifecycle	16
Scheduled Background Work	16
Feed Resilience (FR1–FR8)	18
5. Data Model and Persistence Design	19
The five data layers	19
L0: raw immutable inventory	20
The Durable-Enrichment Identity Contract	21
Migrations and schema versioning	21
SBOM deletion	21

6. Correlation, VEX Overlay, and Risk Scoring	23
Correlation engine	23
Vendor feed taxonomy (CR-4)	24
VEX overlay semantics	25
Risk scoring	25
Threat signals and retroactive re-enrichment	26
7. Security and Trust Model	27
Authentication	27
The artifact trust gate	27
The integrity chain	28
Audit logging	28
Secrets policy	29
Availability protection and input hardening	29
Append-only evidence as a forensic property	29
8. Quality Engineering	30
Test Taxonomy	30
Coverage Governance	31
Static Gates	32
Task-Group Gate Procedure	32
CI Reality	32
End-to-End Gate Scripts	33
9. Development Process and Roadmap	34
Spec-Driven Development with OpenSpec	34
Documentation Organization	35
Release History	35
Active Change: themis-ai-1 (v0.4.0)	35
Scope Guardrail and Forward Roadmap	36
10. Appendices	37
Appendix A: Glossary	37
Appendix B: Configuration Reference	38
Appendix C: Build and Verification Command Reference	40

1. Product Overview

1.1 What Themis Is

Themis is an open-source security-intelligence platform written in Go. It ingests Software Bill of Materials (SBOM) and Vulnerability Exploitability eXchange (VEX) documents, correlates the resulting component catalog against live Common Vulnerabilities and Exposures (CVE) feeds, applies VEX assertions as a non-destructive overlay, watches for newly published CVEs that affect already-ingested components, records human triage decisions, and delivers notifications — all without touching the customer's build system. The Go module is github.com/themis-project/themis.

1.2 Operating Model

Themis deploys as a **single binary backed by PostgreSQL**. There are no agents, no sidecar daemons, and no external message broker: background work (correlation, feed sync, re-enrichment) runs on in-process workers and simple tickers inside the same process. The binary is produced by `make build` as `./bin/themis` and serves an HTTP Application Programming Interface (API) on port 8080 by default. All state lives in one PostgreSQL database; the only mandatory configuration is the connection string, supplied via the `THEMIS_DATABASE_DSN` environment variable.

Property	Value
Language / toolchain	Go 1.25+ (must match <code>go.mod</code>)
Database	PostgreSQL 14+ (sole runtime dependency)
Deployment unit	One binary (<code>./bin/themis</code>), no agents or daemons
API surface	<code>/api/v1/*</code> , <code>X-API-Key</code> auth, async uploads via <code>202 Accepted</code>
Configuration	Optional <code>themis.yaml</code> ; <code>THEMIS_*</code> environment variables override it
Migrations	golang-migrate; single squashed <code>v0.3.0</code> baseline in <code>migrations/</code>

1.3 Capability Summary

Capability	Description
Artifact trust	Schema validation, SHA-256 integrity, deduplication, and provenance checks on every ingested SBOM/VEX document; artifacts must be registered before upload
SBOM parsing	CycloneDX 1.4/1.5/1.6, SPDX (Software Package Data Exchange) 2.3/3.0, and Trivy JSON, normalised to one canonical internal model
Vulnerability correlation	Component catalog matched by Package URL (PURL) and version range against the National Vulnerability Database (NVD), Open Source Vulnerabilities (OSV), and distro feeds (Red Hat, Alpine, Rocky, Wolfi)

Capability	Description
VEX overlay	VEX assertions change status only; raw findings are never deleted, so a revoked VEX safely resurfaces the finding
CVE watch	Background scheduler polls NVD/OSV; new findings are auto-created for matching catalog components
Human triage	Triage API records decisions; each decision auto-generates a <code>themis_generated</code> VEX assertion that survives rescans and re-applies to future ingestions
Notifications	Simple Mail Transfer Protocol (SMTP) and Microsoft Teams delivery with routing rules and digest aggregation
Threat signals	Daily sync of Exploit Prediction Scoring System (EPSS), Known Exploited Vulnerabilities (KEV), ExploitDB, and upstream vendor VEX; retroactive re-enrichment of open findings
Deterministic prioritisation	Layer 1 rules (CVSS, KEV, EPSS, public exploit) set <code>deterministic_level</code> synchronously at ingest time
Blast radius	Asset graph (Product → Microservice → Deployment → Customer) drives a 1.0–2.0× risk multiplier and team routing
VEX export	CycloneDX 1.5+ and OpenVEX export per product version, with an upstream-VEX coverage aggregate
System status	<code>GET /api/v1/status</code> : live counts, severity/state breakdown, top-N ranking, <code>signals_stale</code> and <code>degraded_feeds[]</code>

1.4 Design Philosophy

- **PostgreSQL is the sole source of truth.** There is no secondary datastore, cache tier, or external queue; every durable fact — inventory, findings, judgments, audit records — is a Postgres row, and the binary auto-migrates its schema at startup.
- **Evidence is append-only.** The raw-inventory data layer (L0) is content-addressed and never mutated; SBOM deletion is a soft-delete tombstone (`deleted_at`) that writes an `SBOM_DELETED` audit entry, and VEX overlays adjust `risk_context.effective_state` without deleting the underlying `component_vulnerabilities` rows.
- **Deterministic before AI.** Prioritisation signals (`deterministic_level` , `blast_radius_score`) are computed by rule-based code inside the ingestion pipeline's enrichment stage, before any risk score is written — never by model-driven jobs.
- **Artificial intelligence (AI) is advisory-only.** Under decision D-WRITE-1, AI output never writes `risk_context.effective_state` ; every state change requires a human decision.

Two distinct "layer" vocabularies appear throughout this document. **Code layers** are the Clean Architecture rings — `internal/domain` , `internal/usecase` , `internal/adapters` , `internal/infrastructure` — governed by the inward-only dependency rule. **Data layers L0–L3** classify database content: L0 raw immutable inventory, L1 graph/knowledge context, L2 AI enrichment, L3 human validation. They are unrelated numbering schemes and are never interchangeable.

Scope boundaries evaluators should know: signature verification currently uses a `StubVerifier` (real cosign is deferred), and there is no GitHub Security Advisory adapter, Redis queue, Docker packaging, web UI, or role-based access control. The current schema line is the `v0.3.0` core model with **no in-place upgrade** from pre-`v0.3.0` databases — they must be dropped and recreated.

1.5 Release Status

The current release is **v0.3.11**, the head of a maintenance line (v0.3.2–v0.3.11) on top of the breaking v0.3.0 core-model refactor (2026-06-24). Earlier milestones shipped as Phase 1 (v0.1.0) and Phase 2a (v0.2.0/v0.2.1). One change is active: `themis-ai-1`, targeting **v0.4.0** — a thin slice that adds an advisory CVE Summarizer as a separate `themis-ai` Python service communicating with the Go backend only through a Postgres seam (its own `ai` schema, managed by Alembic, with zero alterations to the core model). Authoritative project status lives in `openspec/STATUS.md`; behavioural specifications live under `openspec/specs/` (18 capabilities).

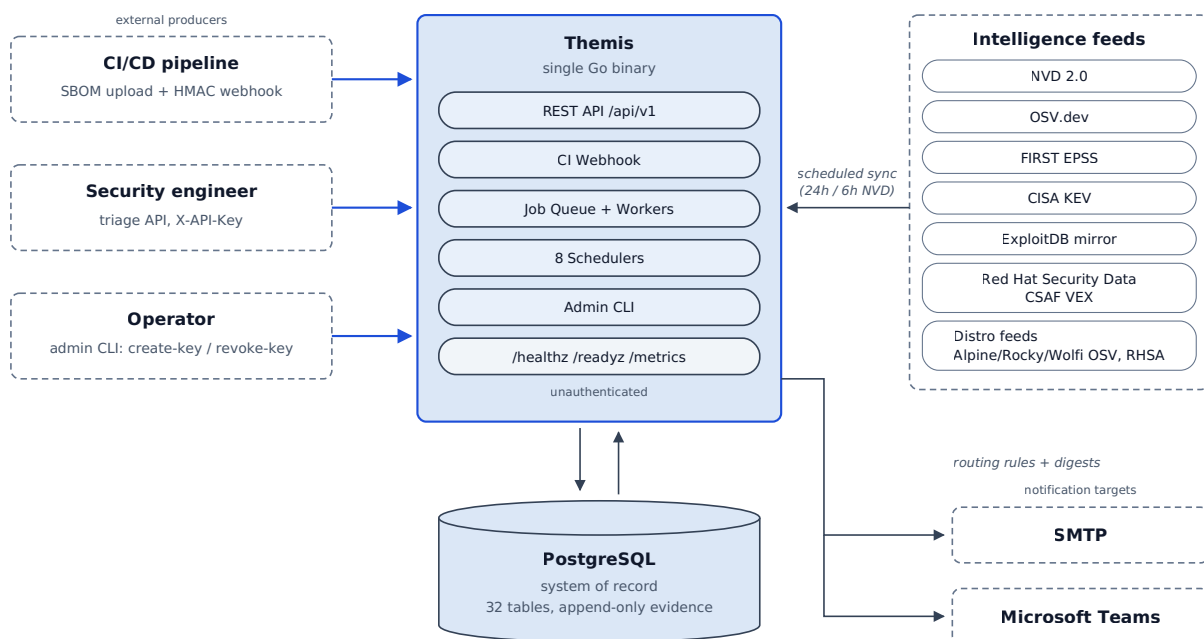


Figure 1 — System context — Themis, its data stores, producers, and external intelligence feeds

2. System Architecture

Themis ships as a single Go binary (`bin/themis`, module `github.com/themis-project/themis`, Go 1.25.0) backed by PostgreSQL. It follows Clean Architecture with an absolute inward Dependency Rule: source-level imports point only toward the domain core, never outward. Two distinct layer vocabularies appear throughout this document and must not be conflated. *Code layers* are the four Clean Architecture rings — `domain`, `usecase`, `adapter`, `infrastructure` — governing Go imports. *Data layers* L0–L3 describe the persistence model (L0/L1 append-only inventory truth, L2 Vulnerability Exploitability eXchange (VEX) intelligence, L3 temporal signals, converging in `risk_context`) and are covered in the data-model chapter. This chapter concerns the code layers.

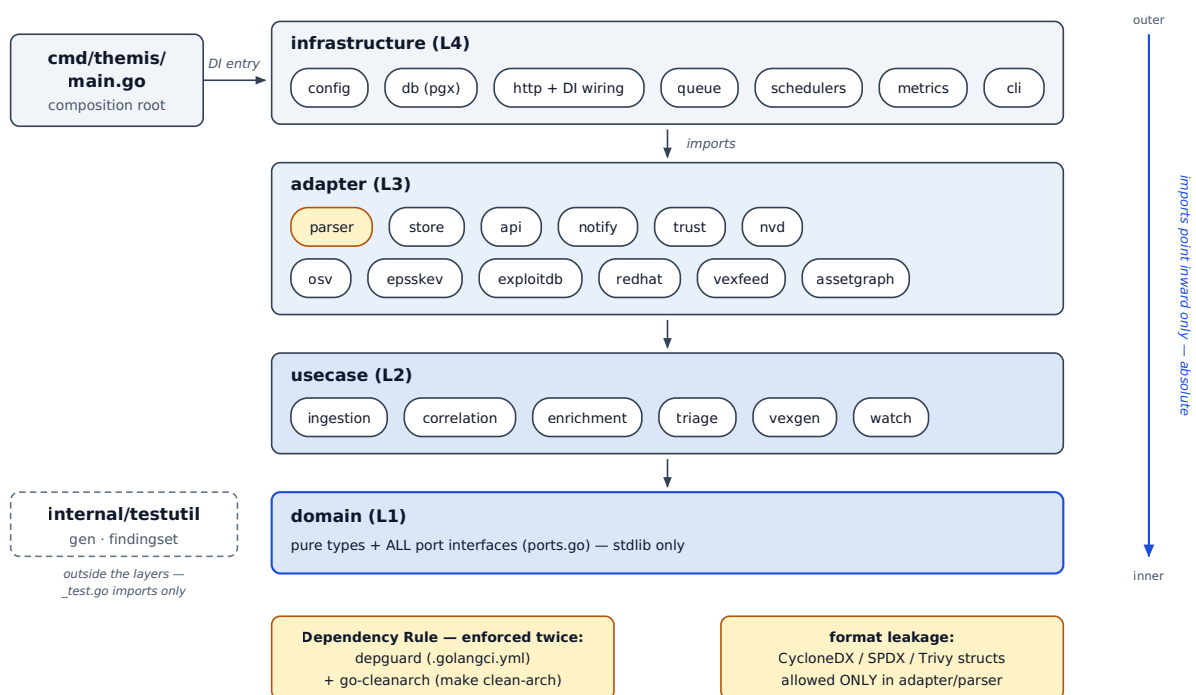


Figure 2 — Clean Architecture code layers and the inward-only Dependency Rule

2.1 The Dependency Rule and the Four Code Layers

Each layer may import only the layers inside it. The allowances are exact, not aspirational — they are encoded as `depguard` rules in `.golangci.yml` and each rule name below is the enforcing rule.

Layer	Path	May import	depguard rule
Domain	<code>internal/domain/</code>	Go standard library only (<code>\$gostd</code>)	<code>domain-stdlib-only</code>
Use case	<code>internal/usecase/</code>	stdlib + <code>internal/domain</code>	<code>usecase-domain-only</code>

Layer	Path	May import	depguard rule
Adapter	<code>internal/adapter/</code>	<code>stdlib</code> + <code>internal/domain</code> + <code>internal/usecase</code>	<code>adapter-inner-only</code>
Infrastructure	<code>internal/infrastructure/</code>	all inner layers	—
Entry point	<code>cmd/themis/</code>	only <code>internal/infrastructure/{config, db, http, http/middleware, metrics, queue}</code>	<code>cmd-infrastructure-only</code>

The use-case layer is transport- and vendor-free by rule: no HTTP, no `pgx`, no Simple Mail Transfer Protocol (SMTP), no Prometheus, no OpenTelemetry (OTel) imports in `internal/usecase/`. Telemetry spans are injected via context and middleware, never constructed inside use cases.

`cmd/themis/main.go` is deliberately thin: `themis admin ...` dispatches to `internal/infrastructure/cli.RunAdmin`; everything else calls `internal/infrastructure/http.Run(ctx)`. The single dependency-injection composition root is `MountAPI` in `internal/infrastructure/http/api_wiring.go`, which constructs every Postgres store, feed client, scheduler, and the ingestion pipeline.

2.2 Dual Mechanical Enforcement

The layering is enforced twice, by independent tools, so a violation cannot land quietly:

- **depguard** — the five rules above in `.golangci.yml`, run under `make lint` (golangci-lint v2). `tools.go` and `_test.go` files are exempt.
- **go-cleanarch** — `make clean-arch` runs `github.com/roblaszczak/go-cleanarch` over `./internal` with the mapping `-domain domain -application usecase -interfaces adapter -infrastructure infrastructure`.

Both run inside `make check` (build + lint + clean-arch + coverage + deadcode), the standard pre-completion gate.

2.3 The Format-Leakage Rule

Software Bill of Materials (SBOM) wire formats are treated as transport, not domain. The `no-format-leakage` depguard rule denies `github.com/CycloneDX/cyclonedx-go`, `github.com/spdx/tools-golang`, and `github.com/anchore/syft` in every Go file except `internal/adapter/parser/**`.

Only `internal/adapter/parser/` may know CycloneDX 1.4–1.6, Software Package Data Exchange (SPDX) 2.3–3.0, or Trivy JSON. Its registry normalises every upload into `CanonicalSBOM`; all other packages — use cases, stores, the API layer — see only `CanonicalSBOM` and canonical components identified by package URL (PURL). Adding a format struct outside the parser fails lint.

2.4 Package Responsibilities

Adapter subpackages (`internal/adapter/`)

Package	Responsibility
<code>nvd</code>	National Vulnerability Database (NVD) 2.0 client with token-bucket rate limiting (default 1.5 requests/s with an API key, 0.15 without, capacity 1 — the limits are load-bearing against NVD's Cloudflare throttle).
<code>osv</code>	OSV.dev (Open Source Vulnerabilities) client plus <code>ComponentFetcher</code> , a live correlation source.
<code>epsskev</code>	FIRST.org Exploit Prediction Scoring System (EPSS) gzip-CSV and CISA Known Exploited Vulnerabilities (KEV) JSON sync (24 h) into threat signals; dispatches re-enrichment.
<code>exploitdb</code>	Exploit-DB GitLab CSV mirror sync into <code>exploit_records</code> .
<code>vexfeed</code>	Vendor-feed machinery with a two-class taxonomy: <code>overlay</code> feeds (Red Hat CSAF VEX → <code>vex_assertions</code>) vs <code>correlation</code> feeds (Alpine/Rocky/Wolfi OSV + RHSA → in-memory correlation index).
<code>redhat</code>	Red Hat Security Data API client for per-CVE VEX overlay.
<code>parser</code>	The sole home of SBOM format libraries; normalises to <code>CanonicalSBOM</code> .
<code>trust</code>	Trust <code>Gate</code> (schema, SHA-256, provenance checks) with <code>StubVerifier</code> , plus a Postgres audit recorder.
<code>notify</code>	SMTP and Microsoft Teams senders, routing rules, digest aggregation, redaction; jobs enqueued via <code>notify.EnqueueSender</code> .
<code>store</code>	All Postgres repositories (one file per aggregate); owns <code>BinarySchemaVersion</code> and schema-shape verification (<code>migrate.go</code>).
<code>assetgraph</code>	Postgres asset graph (Product → Microservice → Deployment → Customer) and the blast-radius multiplier.
<code>api</code>	HTTP handlers, mappers, and authentication under <code>/api/v1</code> .

Use-case packages (`internal/usecase/`)

Package	Responsibility
<code>ingestion</code>	Async pipeline: trust gate → parse → correlate → enrich → notify, with per-stage status transitions and stage spans (<code>domain.StartStage</code>).
<code>correlation</code>	<code>NewCorrelator</code> over <code>CorrelationSource</code> s — the single match path for ingest and watch; merge is distro-authoritative for apk/rpm.
<code>enrichment</code>	VEX overlay and <code>risk_context</code> computation, including <code>ApplyVEX</code> .
<code>triage</code>	Human triage decisions; each decision auto-generates a <code>themis_generated</code> VEX document.

Package	Responsibility
<code>vexgen</code>	Generation of Themis-authored VEX documents.
<code>watch</code>	CVE watch re-correlation against NVD/OSV.

2.5 Key Ports

All port interfaces are defined in the domain package, so outer layers depend on abstractions the core owns. The job-queue ports and job-type constants (`ingest_sbom`, `ingest_vex`, `correlate_vulns`, `reenrich_vex`, `reenrich_signals`, `sync_vex_feed`, `apply_vex_sbom`, `notify`) live in `internal/domain/ports.go`; sibling domain files hold the rest.

Port	Defined in	Implementation today
<code>JobQueue</code> / <code>WorkerPool</code>	<code>internal/domain/ports.go</code>	<code>queue.InProcessQueue</code> (goroutine pool; jobs persisted via <code>queue.NewPostgresJobStore</code> before dispatch).
<code>SignatureVerifier</code>	<code>internal/domain/trust.go</code>	<code>trust.StubVerifier</code> — records trust status without cryptographic verification; real cosign-based crypto is deferred to Phase 3.
<code>Logger</code>	<code>internal/domain/logger.go</code>	zap, wrapped by <code>infrastructure/logging.NewFromZap</code> ; callers use <code>domain.LoggerOrNop</code> for nil safety.
<code>CorrelationSource</code>	<code>internal/domain/ingestion.go</code>	OSV.dev live fetcher and the in-memory distro assertion index (<code>vexfeed</code>).

2.6 Outside the Layers: `internal/testutil`

`internal/testutil/{gen,findingset}` deliberately sits outside the four code layers: `gen` provides `rapid` property-test generators, and `findingset` is the CR-10 golden finding-set diff harness (regenerate goldens with `UPDATE_GOLDEN=1`). Test files are exempt from depguard, so this placement does not weaken the production import graph.

3. Runtime Design and API Surface

3.1 Process model and boot sequence

Themis ships as a single Go binary (`bin/themis`) with two modes, dispatched in `cmd/themis/main.go` : an invocation beginning with `themis admin ...` is routed to `internal/infrastructure/cli.RunAdmin` and exits; any other invocation calls `internal/infrastructure/http.Run(ctx)` and becomes the long-running server. In the code-layer vocabulary (domain / usecase / adapter / infrastructure — distinct from the L0–L3 data layers), both entry points live in the infrastructure layer; `cmd/` is permitted to import only `internal/infrastructure/{config,db,http,http/middleware,metrics,queue}`.

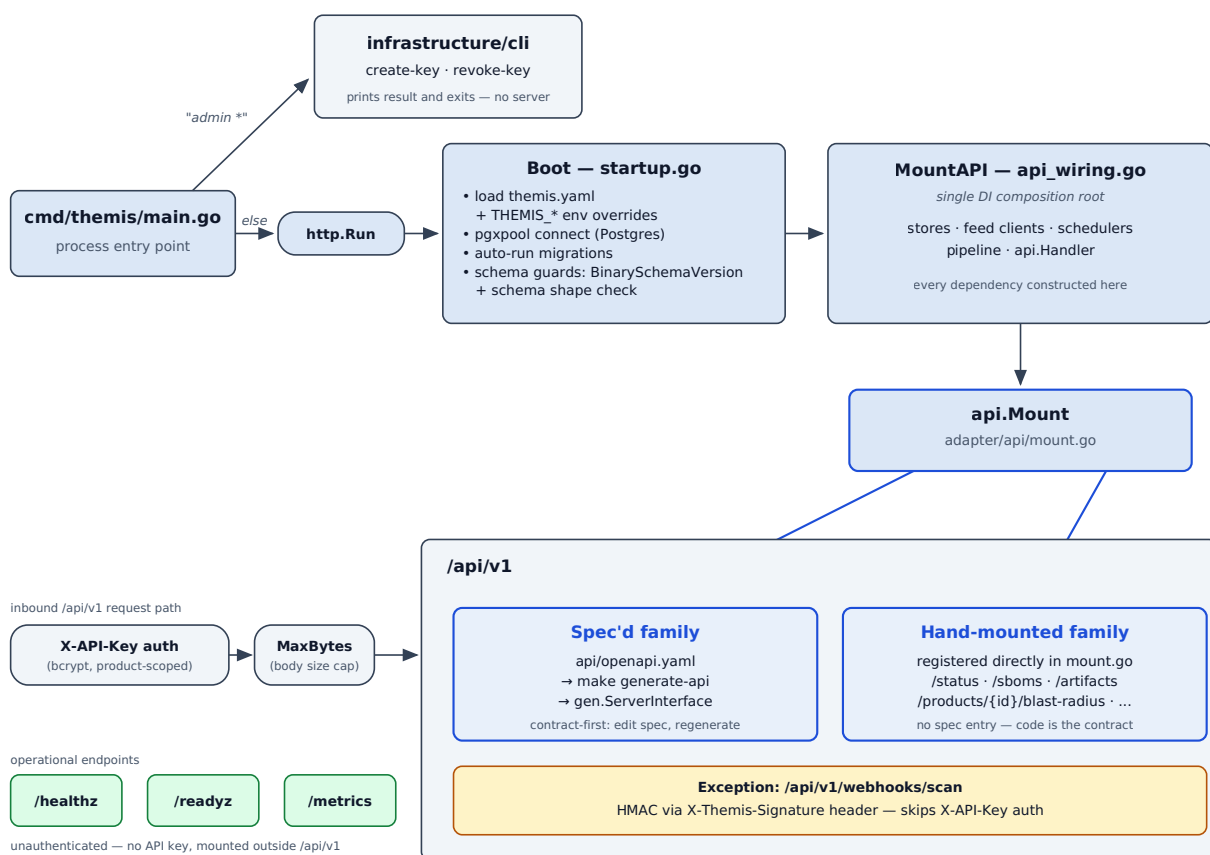


Figure 3 — Boot sequence, dependency-injection composition, and the two API endpoint families

`http.Run` delegates to **Boot** in `internal/infrastructure/http/startup.go`, which executes a fixed sequence:

1. **Configuration load.** `themis.yaml` is read from the current working directory (relocatable via `THEMIS_CONFIG_PATH`) and is optional; every key can be overridden by a `THEMIS_*` environment variable (full mapping in `internal/infrastructure/config/load.go`). Environment always wins over YAML. The only mandatory setting is `THEMIS_DATABASE_DSN`; its absence fails boot with `missing required configuration field: database.dsn`. Secrets (DSN, SMTP password, NVD key, webhook secret) are environment-only by policy and never committed to `themis.yaml`.

2. **Database connection** via `pgxpool` (default `max_pool_size: 20`).
3. **Auto-migration at every boot.** Migrations under `migrations/` (resolved relative to the working directory) are applied automatically; `make migrate-up` exists only for manual use.
4. **Schema guards.** `VerifySchemaVersion` refuses to start if the database schema version is ahead of the binary's `store.BinarySchemaVersion`; `VerifySchemaShape` asserts the v0.3.0 core-model table set and rejects legacy pre-v0.3.0 databases (e.g. `sbom_documents` / `images`) with a "re-initialise your database" message.
5. **Worker pool and server.** The `InProcessQueue` worker pool is constructed, then the HTTP server is started.

Because migrations and `themis.yaml` resolve by working directory, the server must be run from the repository root. Adding a migration also requires bumping `store.BinarySchemaVersion`, or the boot-time version guard misbehaves. After a code change, rebuild the binary *before* resetting the database — an old binary against a fresh database re-creates stale data.

3.2 Dependency-injection composition root

`MountAPI` in `internal/infrastructure/http/api_wiring.go` (~330 lines) is the single dependency-injection (DI) composition root for the whole server. It constructs every Postgres store, feed client, and scheduler, the ingestion pipeline, and the `api.Handler`, then hands them to `api.Mount` (`internal/adapters/api/mount.go`). No other file wires concrete implementations to ports.

3.3 Authentication and unauthenticated surfaces

`api.Mount` places everything under `/api/v1` behind two middlewares: `MaxBytes(upload.max_size_bytes)` (default 52428800 bytes, 50 MiB) and `APIKeyAuth`, which bcrypt-compares the `X-API-Key` header against all active hashes in the `api_keys` table. Keys are product-scoped (`--product-id`) or global admin. One path is exempt: `/api/v1/webhooks/scan` skips API-key auth (a path check inside the middleware) and is instead wrapped by `WebhookAuth`, which verifies an HMAC-SHA256 (hash-based message authentication code) signature in `X-Themis-Signature` against `webhook.secret` / `THEMIS_WEBHOOK_SECRET`. Three endpoints live *outside* `/api/v1` and are unauthenticated (`internal/infrastructure/http/server.go`): `/healthz`, `/readyz` (database ping plus CVE-feed freshness), and `/metrics` (Prometheus).

3.4 Two endpoint families

The API surface is split — non-obviously — into two families:

Family	Definition	Serving path	Contents
Spec'd (16 paths)	<code>api/openapi.yaml</code> ; every operation has an <code>operationId</code> , servers base <code>/api/v1</code>	Generated <code>gen.ServerInterface</code> via <code>gen.HandlerFromMux</code>	SBOM/VEX upload, webhook, ingestions, products/projects/scans, triage, components, CVE watch, config

Family	Definition	Serving path	Contents
Hand-mounted (15 routes)	Registered directly in <code>mount.go</code> ; not in the OpenAPI spec	Explicit <code>r.Get/Post/Delete</code> calls after <code>HandlerFromMux</code>	<code>POST /projects/{id}/versions</code> , <code>POST /products/{id}/artifacts</code> , <code>microservices/deployments/customers</code> , <code>blast-radius</code> , <code>product/project/version-scoped vulnerabilities</code> , <code>version VEX</code> and <code>VEX coverage</code> , <code>GET /status</code> , <code>GET /sboms</code> , <code>GET /products/{id}/sboms</code> , <code>DELETE /sboms/{id}</code>

Adding a spec'd endpoint follows a fixed procedure: (1) edit `api/openapi.yaml`; (2) run `make generate-api` (oapi-codegen v2.7.1 with `api/oapi-codegen.yaml`, package `gen`, `chi-server` + `models` + embedded spec) to regenerate the checked-in `internal/adapters/api/gen/api.gen.go`; (3) implement the new method on `*api.Handler` in a `handlers_*.go` file — compilation breaks until then, enforced by `var _ gen.ServerInterface = (*webhookServer)(nil)` in `mount.go`; (4) if new dependencies are needed, add a field to `api.Dependencies` in `server.go` and wire the concrete store in `MountAPI`. New hand-mounted routes are instead registered in `mount.go`; contributors must decide which family a new endpoint joins.

3.5 API conventions

- **Cursor pagination.** List endpoints accept a `cursor` query parameter and return `next_cursor`.
- **Idempotency.** Mutating endpoints accept an `Idempotency-Key` header; uploading the same bytes for the same `(image_digest, checksum_sha256)` pair is idempotent and returns the existing scan without re-correlating.
- **Asynchronous semantics.** SBOM (software bill of materials) and VEX (Vulnerability Exploitability eXchange) uploads return `202 Accepted` with an `ingestion_id`. The entire pipeline — including Layer-1 deterministic scoring, which runs inside the enrichment stage before any risk score is written — executes on the background job queue after the 202 is returned.

A `202 Accepted` only means the job was queued — it is not success. Clients must poll `GET /api/v1/ingestions/{id}` until a terminal state (`NOTIFIED`, `COMPLETED`, `FAILED`, `REJECTED`); `stage_detail` is the authoritative failure reason.

3.6 Error envelope

Every endpoint returns errors in one envelope — `{"error":{"code","message","hint"}}` — written by `WriteCatalogError / MapError` in `internal/adapters/api/errors.go`. Raw Postgres or Go error strings never appear in response bodies. The catalogue (14 stable codes) comprises: `SBOM_NOT_FOUND`, `PRODUCT_NOT_FOUND`, `PROJECT_NOT_FOUND`, `VERSION_CONFLICT`, `IMAGE_NOT_FOUND`, `CUSTOMER_NOT_FOUND`, `CANNOT_DELETE_LATEST_SBOM`, `DUPLICATE_MICROSERVICE`, `DUPLICATE_CUSTOMER`, `INVALID_SBOM_FORMAT`, `INVALID_REQUEST`, `MISSING_API_KEY`, `INVALID_API_KEY`, `INTERNAL_ERROR`.

3.7 Command-line surface

The entire command-line interface (CLI) lives in `internal/infrastructure/cli/admin.go`:

Command	Flags	Behaviour
<code>./bin/themis</code>	—	Runs the server; requires <code>THEMIS_DATABASE_DSN</code> .
<code>./bin/themis admin create- key</code>	<code>--name</code> (default <code>cli-created</code>), exactly one of <code>--admin</code> or <code>-- product-id UUID</code> , optional <code>-- expires</code> (e.g. <code>90d</code> , <code>24h</code>)	Prints <code>key_id=</code> and the raw <code>api_key=</code> once; only the bcrypt hash is stored. Supplying both or neither scope flag is an error.
<code>./bin/themis admin revoke- key</code>	<code>--key-id ID</code>	Marks the key revoked; unknown IDs return an error.

There is no other CLI surface. All operational actions beyond key management — ingestion, triage, deletion — go through the HTTP API, preserving the audit and immutability guarantees that manual SQL bypasses.

4. Ingestion Pipeline and Background Processing

4.1 Transport-Agnostic Ingestion

Themis accepts Software Bill of Materials (SBOM) and Vulnerability Exploitability eXchange (VEX) documents over two transports: direct authenticated upload (`POST /api/v1/sboms` and the VEX upload endpoint, guarded by `X-API-Key`) and the CI-facing webhook `POST /api/v1/webhooks/scan` (guarded by HMAC-SHA256 via `X-Themis-Signature`). Both transports converge on the same use-case entry points — `IngestSBOM` and `IngestVEX` in `internal/usecase/ingestion/service.go` — so pipeline behavior is identical regardless of how a document arrives. In code-layer terms (domain / usecase / adapter / infrastructure), the transport handlers live in `internal/adapter/api/` ; the pipeline itself is a usecase that depends only on domain ports.

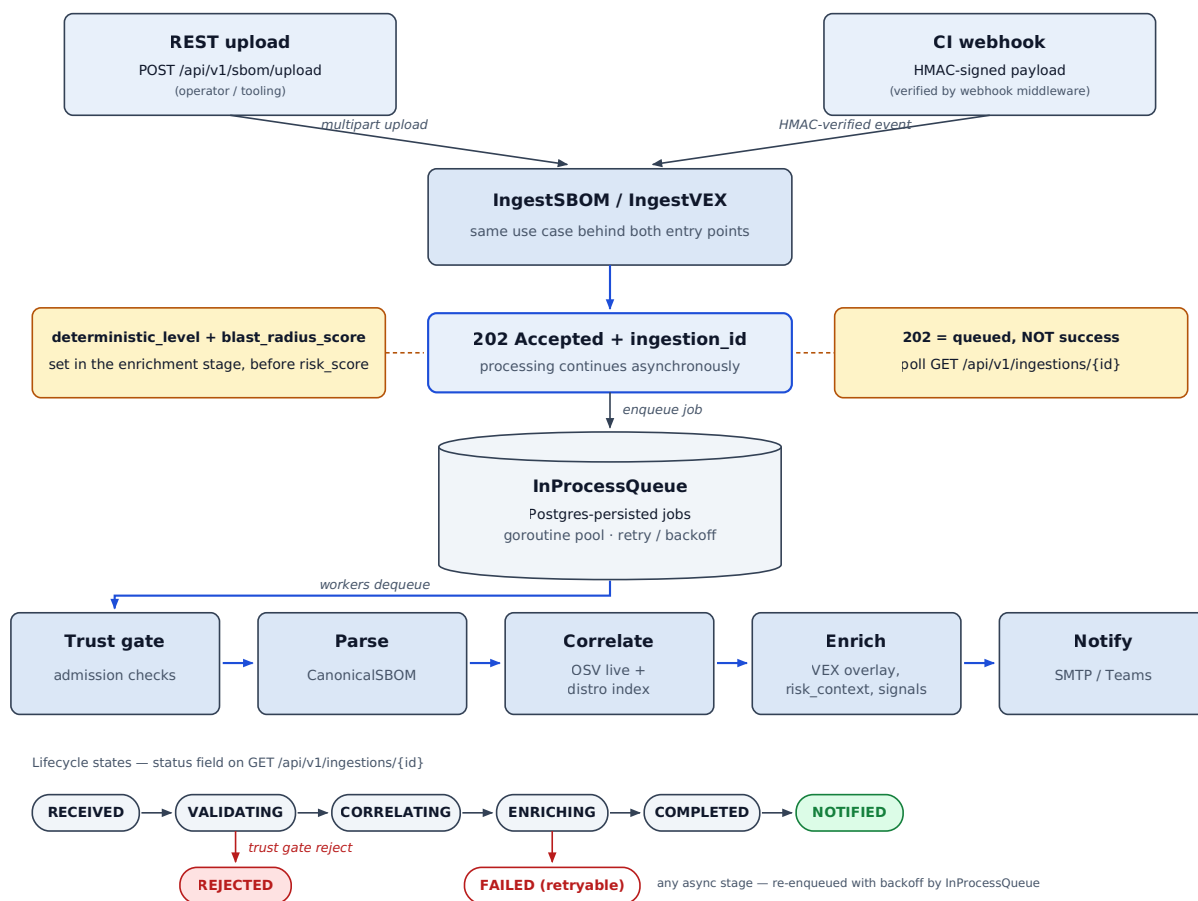


Figure 4 — Asynchronous ingestion pipeline: entry points, queue, stages, and lifecycle states

4.2 Asynchronous by Default: the Job Queue

All ingestion work is dispatched through the `domain.JobQueue` port, implemented by `queue.InProcessQueue` (`internal/infrastructure/queue/inprocess.go`), which also satisfies `domain.WorkerPool`. Jobs are persisted to PostgreSQL via `queue.NewPostgresJobStore` before channel

dispatch to the goroutine pool, so an accepted job survives inspection even though execution is in-process. Failed jobs retry with exponential backoff. The pool is tuned by three configuration keys: `worker.pool_size` (example default 10), `worker.max_retry` (example default 3), and `worker.base_delay`. A Redis-backed queue is an explicit Phase 3 substitution behind the same port.

The job handler is injected late: `MountAPI` (`internal/infrastructure/http/api_wiring.go`) calls `SetHandler` with a single switch on `job.Type`. Job-type constants live in `internal/domain/ports.go`:

Job type	Routed to
<code>ingest_sbom</code> , <code>ingest_vex</code> , <code>correlate_vulns</code> , <code>reenrich_vex</code> , <code>sync_vex_feed</code>	<code>ingestion.JobHandler</code> (default branch)
<code>notify</code>	<code>notify.JobHandler</code>
<code>reenrich_signals</code>	signals re-enrichment handler
<code>apply_vex_sbom</code>	<code>enrichmentSvc.ApplyVEX</code>

4.3 Pipeline Stages and Lifecycle

An accepted SBOM job runs five stages — trust gate, parse, correlate, enrich, notify — each wrapped in a stage span (`domain.StartStage`) and reflected in a persisted status transition. The trust gate (`adapter/trust.Gate`) performs schema, SHA-256, and provenance checks; parsing is confined to `internal/adapter/parser/` and emits `CanonicalSBOM`; correlation runs the unified `correlation.NewCorrelator` over its sources; enrichment applies the VEX overlay; notification enqueues a `notify` job.

Lifecycle statuses are constants in `internal/domain/ingestion.go`: `RECEIVED` → `VALIDATING` → `CORRELATING` → `ENRICHING` → `COMPLETED` → `NOTIFIED`. A trust-gate or schema rejection terminates at `REJECTED`; transient stage errors land in `FAILED`, which is retryable under the queue's backoff policy.

Deterministic scoring guarantee (AC-20, as implemented). The risk-model inputs `deterministic_level` (scoring Layer 1) and `blast_radius_score` (scoring Layer 2) are computed inside the pipeline's enrichment stage, before any `risk_score` is written — never deferred to a separate follow-up job. Because the whole pipeline runs asynchronously, these fields are not yet readable at the moment the `202 Accepted` returns; they are present once the ingestion reaches a terminal state, and the signals re-enrich job refreshes them as feeds update. Note these are risk-scoring layers, distinct from both the code layers and the L0–L3 data layers.

4.4 Scheduled Background Work

Eight schedulers, all in `internal/infrastructure/http/*_scheduler.go`, share one pattern: nil-safe construction, one immediate run at startup, then a `time.Ticker` loop, logging through `domain.Logger` and recording success/failure via `domain.FeedHealthRecorder`. The triage-expiry scheduler is the one exception to the immediate run — its first pass fires only after its one-hour ticker interval.

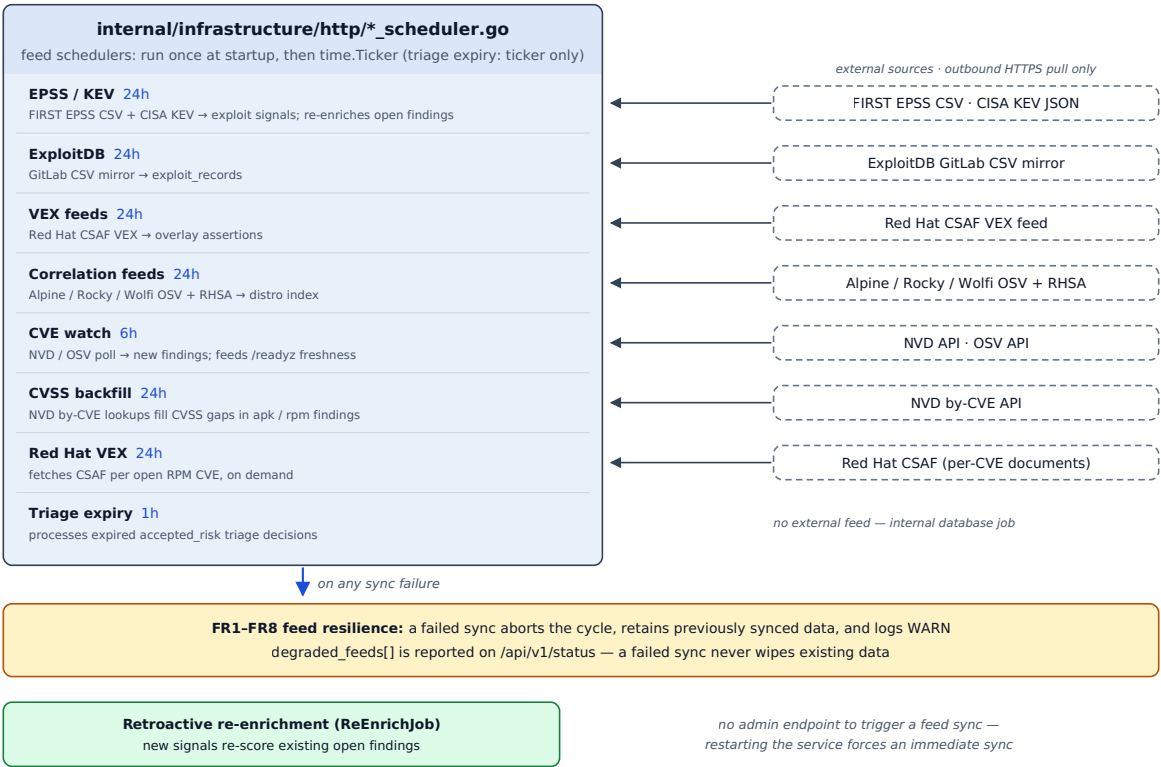


Figure 5 — Background schedulers, their cadences, and the feed-resilience contract

Scheduler	Purpose	Interval
StartEPSSKevScheduler	Exploit Prediction Scoring System (EPSS) CSV + CISA Known Exploited Vulnerabilities (KEV) JSON → threat signals	24 h
StartExploitDBScheduler	ExploitDB CSV mirror → exploit_records	24 h
StartVEXFeedScheduler	Vendor overlay VEX feeds → vex_assertions	24 h
StartCorrelationFeedScheduler	Alpine/Rocky/Wolfi OSV + RHSA correlation feeds → in-memory index	24 h
StartWatchScheduler	National Vulnerability Database (NVD) / Open Source Vulnerabilities (OSV) CVE watch; updates the CVEFeedSuccess atomic consumed by /readyz freshness	6 h
StartCVSSBackfillScheduler	Fills missing Common Vulnerability Scoring System (CVSS) data for apk/rpm findings from NVD by-CVE (CR-5)	configured ticker
StartRedHatVEXScheduler	On-demand Red Hat Security Data API lookups per open RPM CVE → vex_assertions	configured ticker
StartTriageExpiryScheduler	Expires time-boxed triage decisions	1 h

No admin sync-trigger endpoint exists, by design. Every feed scheduler runs once immediately at process start, so the supported way to force a feed refresh is to restart the binary. Do not add an ad-hoc trigger endpoint without a spec change.

4.5 Feed Resilience (FR1–FR8)

Feed synchronization is governed by eight verification rules ([docs/current-changes/verification.md](#), rows FR1–FR8) with one shared invariant: a failed or suspect sync aborts, retains all previously loaded data, and logs a WARNING — it never wipes state and never blocks ingestion. Concretely: an unreachable EPSS endpoint leaves prior `intelligence_signals` untouched (FR1); a truncated EPSS CSV (under 50% of the prior row count) aborts with a row-count comparison logged (FR2); malformed KEV JSON retains previous KEV data (FR3); an empty ExploitDB CSV deletes nothing (FR4); vendor-feed HTTP 429 triggers exponential-backoff retry (FR5); a CSAF advisory missing `productStatus` is skipped while the rest of the sync proceeds (FR6); EPSS values outside [0.0, 1.0] are rejected row-wise (FR7); signals older than their 25-hour TTL are flagged `stale=true` (FR8).

Degradation is observable, never silent. Per-feed health flows through `domain.FeedHealthRecorder` into a `degraded_feeds[]` array on `GET /api/v1/status` (with `signals_stale` for TTL breaches), so operators see every degraded feed from one API call rather than only EPSS/KEV staleness.

5. Data Model and Persistence Design

Themis persists everything in a single PostgreSQL database whose schema is organized as a five-layer data model. These *data layers* (L0–L3, with L1 split into three sub-layers) describe how information gains interpretation as it moves upward, and are distinct from the four *code layers* (domain/usecase/adaptor/infrastructure) described in the architecture chapter. Each data layer has a different mutation discipline, which is the central design idea: raw evidence is never edited, and human judgment is never lost.

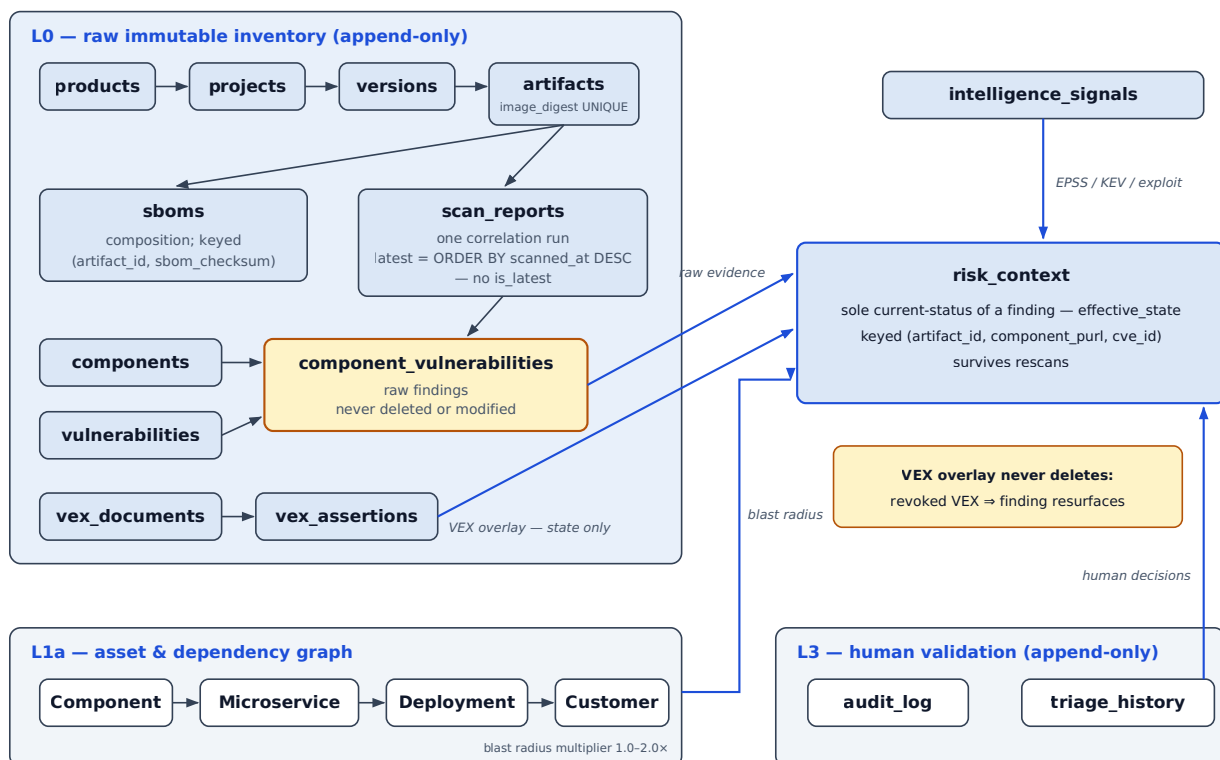


Figure 6 — Data layers and the durable-identity convergence into risk_context

5.1 The five data layers

Layer	Content	Mutation discipline	Status
L0	Raw inventory: artifacts, Software Bills of Materials (SBOMs), scan findings, vulnerability catalog, Vulnerability Exploitability eXchange (VEX) documents	Append-only, content-addressed by SHA-256; never mutated or deleted	Shipped (v0.3.0)
L1a	Asset graph (<code>asset_graph_nodes</code> / <code>asset_graph_edges</code>): Product → Microservice → Deployment → Customer	Mutable registry	Shipped

Layer	Content	Mutation discipline	Status
L1b	Security knowledge graph	—	Planned (Phase 2b+)
L1c	Embeddings (pgvector)	—	Planned (Phase 2b+)
L2	AI enrichment outputs	Immutable per <code>(worker_id, input_hash)</code> ; a re-run inserts a new row	Planned (active change <code>themis-ai-1</code> , v0.4.0)
L3	Human validation: <code>triage_history</code> , <code>audit_log</code> , approvals	Append-only	Shipped

5.2 L0: raw immutable inventory

The ownership spine is `products` → `projects` → `versions` → `artifacts`. `versions.project_id` is NOT NULL; creating a product auto-creates a default project and a "latest" version so callers need not manage the hierarchy explicitly. The `artifacts` table is the v0.3.0 merge of the former artifacts and images tables, and `artifacts.image_digest` is globally UNIQUE — exactly one row exists per container-image digest, and re-registering a digest idempotently returns the same artifact.

v0.3.0 deliberately splits what earlier schemas conflated into two tables:

- `sboms` — an uploaded *composition* document, keyed by `(artifact_id, sbom_checksum)`. Re-uploading identical bytes is idempotent; a divergent document for the same artifact adds a new row.
- `scan_reports` — one *correlation run*'s findings. An artifact accumulates N scan reports over time.

"Latest scan" is defined purely as `ORDER BY scanned_at DESC`. There is deliberately no `is_latest` or `supersedes_id` column: derived-state columns invite skew under concurrent writes, whereas an ordering query cannot disagree with the data. Status endpoints reflect only the latest scan per artifact via the `v_latest_findings` view.

Composition detail lives in `components`, `component_versions`, and `dependency_relationships`. The vulnerability side comprises the `vulnerabilities` catalog and `component_vulnerabilities` — the per-scan raw finding, denormalized with a version-qualified Package URL (`component_purl`) and normalized `cve_id` (Common Vulnerabilities and Exposures identifier). VEX inputs are stored as `vex_documents` (referencing artifacts) fanned out into `vex_assertions`.

VEX is an overlay, never a delete: a VEX assertion changes `risk_context.effective_state` only, while rows in `component_vulnerabilities` are preserved forever. If a VEX statement is revoked, the finding resurfaces automatically.

5.3 The Durable-Enrichment Identity Contract

Scan-scoped rows are disposable evidence; judgments must outlive them. Five tables — `risk_context`, `triage_history`, `remediation_actions`, `intelligence_signals`, and `runtime_exposures` — therefore key on the triple `(artifact_id, component_purl, cve_id)` rather than on any scan identifier. A rescan of the same artifact produces new `scan_reports` and `component_vulnerabilities` rows, but every prior triage decision re-attaches to the recurring finding by identity. Deleting a scan via SQL removes findings but, by design, does not remove durable judgments.

`risk_context` is the sole convergence point: the single source of truth for a finding's current status. Its `effective_state` enumeration is `DETECTED`, `SUPPRESSED`, `CONFIRMED`, `IN_TRIAGE`, `ACCEPTED_RISK`, `FALSE_POSITIVE`, `RESOLVED`. Every human triage decision (L3) auto-generates a `vex_document` with `source=themis_generated` that re-applies on future ingestions of the same `(component_purl, cve_id)`. Until Phase 2c, AI output (L2) is advisory-only and never writes `effective_state`; state changes require a human (decision D-WRITE-1).

5.4 Migrations and schema versioning

The schema ships as a single squashed baseline, `migrations/000001_v030_baseline` (up/down pair), creating all 32 tables; the pre-v0.3.0 chain of 19 migrations no longer exists. Migrations run via `golang-migrate`, compiled **with** `-tags postgres` (the driver is not registered otherwise), either through `make migrate-up` (requires `THEMIS_DATABASE_DSN` exported in the same shell) or automatically at server boot.

Two guards keep binary and database in lock-step:

- `BinarySchemaVersion` — a constant in `internal/adapter/store/migrate.go` (currently `1`) recording the highest migration embedded in the binary. It **must be bumped with every new migration**; startup refuses a database migrated beyond what the binary knows. The planned `themis-ai-1` change adds backend migration `000002` (its separate `ai` schema is owned by the Python service via Alembic, not `golang-migrate`).
- `VerifySchemaShape` — a startup guard in `internal/adapter/store/schema.go` that asserts all expected core-model tables exist and no legacy pre-v0.3.0 tables (`sbom_documents`, `images`) remain, failing loudly with a re-initialise hint (D13 schema-skew guard).

There is no in-place upgrade from a pre-v0.3.0 database. The supported path is drop and recreate: `DROP SCHEMA public CASCADE; CREATE SCHEMA public;` (or `dropdb / createdb`) followed by `make migrate-up`. Prior triage decisions do not carry over.

5.5 SBOM deletion

Consistent with L0 immutability, SBOMs support soft-delete only. `DELETE /api/v1/sboms/{id}` sets a `deleted_at` tombstone; underlying rows are never hard-deleted (acceptance criterion AC-22). Deleting the artifact's latest scan requires an explicit `force=true` query parameter, and every deletion writes an

`audit_log` entry with action `SBOM_DELETED`. Manual SQL deletes in production are prohibited because they bypass both the audit trail and the immutability guarantees.

6. Correlation, VEX Overlay, and Risk Scoring

This chapter describes how Themis turns a normalized component catalog into scored, triage-aware findings. The pipeline stage sequence is: correlate (produce raw findings), overlay Vulnerability Exploitability eXchange (VEX) assertions, then score. In code-layer terms the logic lives in the *usecase* layer (`internal/usecase/correlation`, `internal/usecase/enrichment`) with feed I/O in the *adapter* layer (`internal/adapter/osv`, `internal/adapter/vexfeed`, `internal/adapter/epsskev`, `internal/adapter/exploitdb`). In data-layer terms, raw findings and VEX documents land in L0 (append-only inventory: `component_vulnerabilities`, `vex_documents`, `vex_assertions`), human triage is recorded in L3 (`triage_history`, `audit_log`), and everything converges into `risk_context` — the sole source of truth for a finding's current status.

6.1 Correlation engine

Findings are produced exclusively by Package URL (PURL) matching. A single `correlation.NewCorrelator(logger, sources ...domain.CorrelationSource)` (`internal/usecase/correlation/correlator.go`) fans each component out to its sources in order: the live OSV.dev (Open Source Vulnerabilities) API and an in-memory distro index built from vendor feeds. Components without a `purl` are skipped entirely.

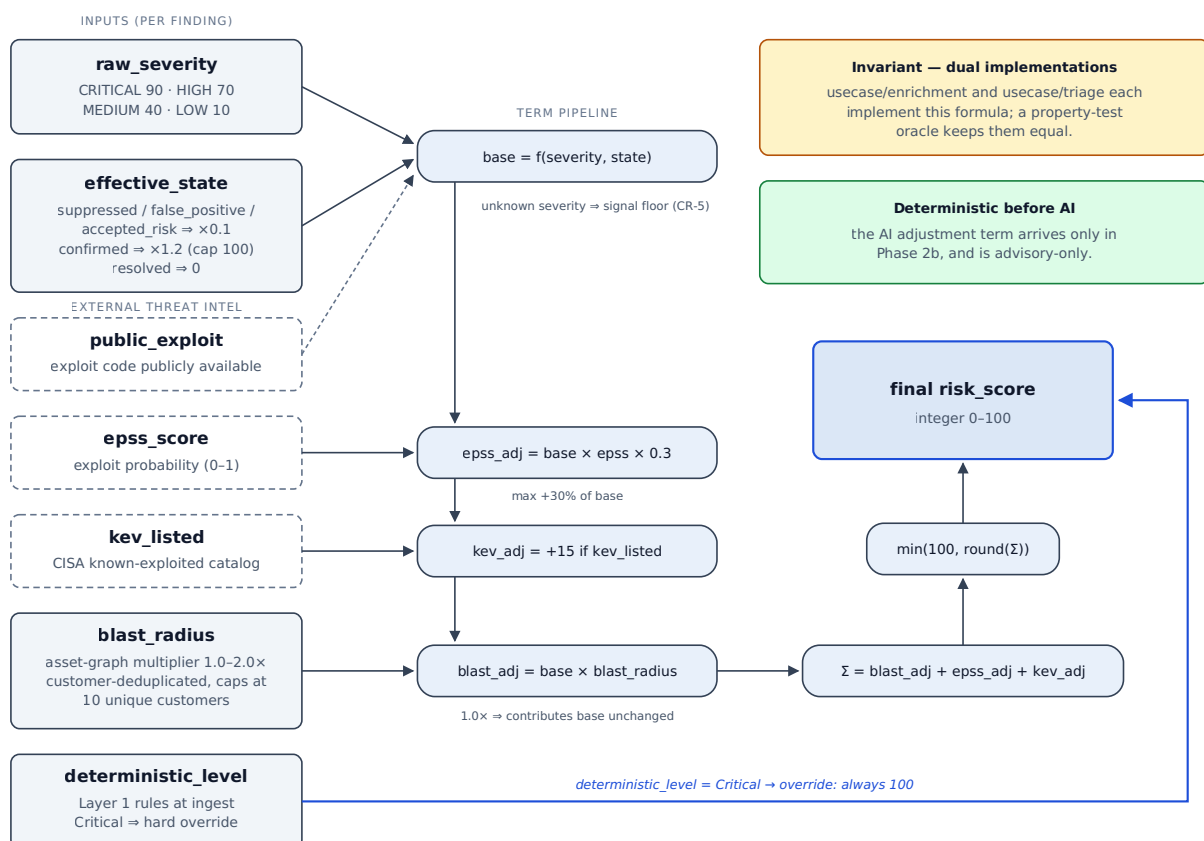


Figure 7 — Deterministic risk-scoring pipeline (Phase 2a composite formula)

The CycloneDX embedded `vulnerabilities` array is deliberately **not** ingested. Themis re-derives findings from its own sources at ingest time; scanner-supplied verdicts are treated as untrusted transport data.

OSV queries require translating PURL type to OSV ecosystem — the mapping lives in `internal/adaptor/osv/ecosystem.go`:

PURL type	OSV ecosystem	Notes
<code>apk</code> , <code>alpine</code>	Alpine	Fixed version is exclusive: range is <code>[introduced, fixed)</code> ; installed == fixed means <code>not_affected</code>
<code>deb</code> , <code>debian</code>	Debian	—
<code>ubuntu</code>	Ubuntu	—
<code>npm</code> , <code>maven</code> , <code>pypi</code> , <code>go</code> , <code>cargo</code> , <code>nuget</code> , <code>gem</code> , <code>composer</code> , <code>hex</code> , <code>pub</code>	Language ecosystems (e.g. <code>crates.io</code> for <code>cargo</code>)	—
<code>rpm</code> , <code>generic</code> , <code>oci</code>	<i>Unmapped by design</i>	OSV lookup skipped; RHEL-family SBOMs get sparse automatic findings, partially filled by the distro/RHSA feeds below

Further matching rules, each the product of a documented defect fix:

- **Distro-authoritative merge (CR-3)**: when both a distro feed and OSV.dev report the same Common Vulnerabilities and Exposures (CVE) identifier for an `apk` / `rpm` component, the distro verdict outranks OSV.dev — distro maintainers know their backports.
- **OSV GIT-type ranges are skipped** (v0.3.7): commit hashes must never become version bounds.
- **CVE identifiers are normalized** via `domain.NormalizeCVEID` (e.g. `ALPINE-CVE-*` becomes `CVE-*`) so dedup keys align across sources.

6.2 Vendor feed taxonomy (CR-4)

The `vexfeed` subsystem enforces a strict two-class taxonomy. Confusing the classes is a correctness bug: an overlay feed must never manufacture findings, and a correlation feed's advisories are evidence, not suppression.

Built-in feed	Class	Effect
<code>rhel-vex</code> (Red Hat CSAF VEX, <code>rhel_vex_url</code>)	overlay	Writes <code>vex_assertions</code> only; never creates findings
<code>rhel-csaf</code> (Red Hat Security Advisories)	correlation	Creates findings, provenance <code>rhsa</code>
<code>alpine</code> , <code>rocky</code> , <code>wolfi</code> (OSV bulk)	correlation	Creates findings, provenance <code>distro_osv</code>

Provenance constants are defined in `internal/domain/finding_source.go`. User configuration under `vexfeed.feeds:` is a *delta list* merged by name over the built-in registry; entries accept `type: url|zip-osv|csaf-dir`, `class: correlation|overlay`, and `enabled: false` to disable a built-in. `rhel_url` is a deprecated alias retained for one release. A failed feed emits a structured warning and appears in `degraded_feeds[]` on `GET /api/v1/status`; previously cached data is always retained (Feed Resilience rules FR1–FR8: abort, retain, warn — never wipe).

6.3 VEX overlay semantics

VEX overlay, never delete. Assertions flip `risk_context.effective_state` only; raw rows in `component_vulnerabilities` are preserved forever. If a VEX assertion is revoked, the finding resurfaces automatically.

Vendor VEX is authoritative for backports (acceptance criterion AC-19): once a vendor VEX statement matches a component/CVE pair, the upstream CVE version range is never consulted again — this is what makes patched-but-version-old distro packages report correctly. When multiple assertions target the same finding, export precedence is `themis_generated` > `manual / vendor` > `ai_generated` > `upstream_vendor`. Every human triage decision auto-generates a `source=themis_generated` VEX document plus a durable `risk_context` verdict keyed `(artifact_id, component_purl, cve_id)`, so analyst judgments survive rescans and re-apply to future ingestions of the same pair.

6.4 Risk scoring

The Phase 1 formula maps severity and effective state to a base score:

```
base: CRITICAL=90 HIGH=70 MEDIUM=40 LOW=10
SUPPRESSED / FALSE_POSITIVE / ACCEPTED_RISK -> base x 0.1
CONFIRMED -> base x 1.2 (cap 100); RESOLVED -> 0; DETECTED -> base
```

Phase 2a replaces it with a composite (a breaking change — severity-only thresholds built on Phase 1 scores are invalid):

```
base      = f(raw_severity, effective_state)      # Phase 1 formula
layer1    = 100 if deterministic_level == Critical else base
epss_adj  = base x (1 + epss_score x 0.3)         # +0..30%; 0 when NULL
kev_adj   = +15 if kev_listed else 0
blast_adj = base x blast_radius_score             # multiplier 1.0-2.0x
final     = min(100, layer1 + epss_adj + kev_adj + blast_adj)
```

Tested invariants: scores stay in [0,100]; `deterministic_level=Critical` forces 100 regardless of other terms; suppression is monotonically decreasing. The blast-radius multiplier is derived from the asset graph (Component to Microservice to Deployment to Customer), deduplicated by unique Customer — three deployments for one customer yield 1.0, not 1.2 — and caps at 10 unique customers

(`intelligence.blast_radius_cap` / `THEMIS_INTELLIGENCE_BLAST_RADIUS_CAP`). It defaults to `1.0` until an asset graph is registered.

`deterministic_level` and `blast_radius_score` are populated by the ingestion pipeline's enrichment stage itself, before `risk_score` is written — never deferred to a separate follow-up job (AC-20, as implemented). They become readable once the ingestion reaches a terminal state, not at the `202 Accepted`.

6.5 Threat signals and retroactive re-enrichment

EPSS (Exploit Prediction Scoring System) and KEV (Known Exploited Vulnerabilities) sync daily, alongside an ExploitDB mirror; all signal feeds poll on a 24-hour ticker (not cron), starting with a sync at process startup — there is no admin trigger endpoint. New signal data is applied retroactively: the `ReEnrichJob` (batch entry point `Handler.ReEnrichSignalsBatch` in `internal/usecase/enrichment/reenrich_signals.go`) refreshes signal fields and recomputes risk scores for open findings without requiring a re-upload. Consequently, `epss_score`, `kev_listed`, and `exploit_public` on the findings enrichment object remain empty until the first feed sync and back-fill completes. Sync health is observable via `themis_epsskev_sync_total{feed,status}`, `themis_epsskev_stale`, `themis_vexfeed_sync_total`, `themis_vexfeed_assertions_total{feed,match_type}`, and `themis_reenrichjob_batches_total` on `/metrics`.

7. Security and Trust Model

Themis treats security controls as a deliberately scoped surface: strong controls where evidence integrity is at stake (the trust gate, the audit trail, the append-only Layer 0 data), simple but explicit controls at the network edge (API keys, webhook HMAC), and documented deferrals for everything else. Role-Based Access Control (RBAC), OpenID Connect (OIDC), request rate limiting, and real cryptographic signature verification are all Phase 3 items by explicit scope decision — the project rule is verbatim: "Do not implement Phase 3 features (rate limiting, cosign, CI/CD, Docker, UI, Redis, RBAC) without explicit user direction." In this chapter, "code layers" refers to the Clean Architecture packages (domain/usecase/adapters/infrastructure); "data layers L0-L3" refers to the evidence model in PostgreSQL.

7.1 Authentication

All authentication is API-key based. Every route under `/api/v1` passes through `apimiddleware.APIKeyAuth` (`internal/adapters/api/middleware/auth.go`), which reads the `X-API-Key` header and bcrypt-compares the presented key against *all* active hashes in the `api_keys` table (`key_hash TEXT NOT NULL UNIQUE`; raw keys are never stored). Keys carry a `scopes` array (product-scoped in production), an optional `expires_at`, and a `revoked_at` tombstone; expired or revoked keys are skipped during comparison. A missing or unmatched key yields `401` with the standard error envelope. Key lifecycle is command-line only — there is no key-management HTTP endpoint:

```
./bin/themis admin create-key --name N [--product-id UUID | --admin] [--expires 90d]
./bin/themis admin revoke-key --key-id ID
```

`create-key` prints `key_id=` and the raw `api_key=` exactly once; only the bcrypt hash persists. `--admin` is intended for local development; production keys should be product-scoped via `--product-id`. Health endpoints (`/healthz`, `/readyz`, `/metrics`) live outside `/api/v1` and are intentionally unauthenticated.

Webhook authentication

The single exception to API-key auth is `/api/v1/webhooks/scan`: the API-key middleware detects the path and passes it through, and the `webhookServer` wrapper applies `WebhookAuth` instead. Continuous Integration (CI) callers must send `X-Themis-Signature`, a hex-encoded HMAC-SHA256 (hash-based message authentication code) of the raw request body, keyed by the secret from `THEMIS_WEBHOOK_SECRET` (config key `webhook.secret`). Verification uses constant-time `hmac.Equal`; an unconfigured secret fails closed with `401 webhook secret not configured`.

7.2 The artifact trust gate

Every Software Bill of Materials (SBOM) or Vulnerability Exploitability eXchange (VEX) upload passes through `trust.Gate.Evaluate` (`internal/adapters/trust/gate.go`) before any Layer 0 write. The gate runs a fixed pipeline: SHA-256 checksum comparison against the caller-supplied expected checksum;

document schema validation; spec-version validation; deduplication (a duplicate `(image_digest, checksum_sha256)` returns `200` with the existing artifact's ID — idempotent by design); integrity-chain validation; provenance and supplier checks; and finally signature verification. Behavior of the soft checks depends on the trust policy, selected per upload via `?trust_policy=` or globally via `THEMIS_TRUST_DEFAULT_POLICY` (default `standard`).

Policy	Provenance (<code>ci_job_id</code> , <code>ci_pipeline_url</code> , <code>supplier_identity</code>)	Unknown supplier	Unsigned artifact
<code>strict</code>	Missing fields reject (<code>422</code>)	Reject (<code>422</code>)	Reject (<code>422</code>)
<code>standard</code>	Missing fields warn; accepted with <code>ARTIFACT_WARNING</code> audit action	Warn	Accept, recorded as <code>unsigned</code>
<code>permissive</code>	Not checked	Not checked	Accept

7.3 The integrity chain

Themis records a referential chain across artifacts (a permanent invariant): a VEX document must reference an existing SBOM checksum, and an SBOM must reference a registered container image digest. The gate enforces this before acceptance — an SBOM whose `image_digest` has no `artifacts` row is rejected with `image not found – ingest parent first`, and a VEX whose `sbom_checksum` is unknown is rejected analogously. Cryptographic verification of that chain is a port, not yet an implementation: `domain.SignatureVerifier` is satisfied today by `StubVerifier` (`internal/adaptor/trust/stub_verifier.go`), which classifies artifacts as `unsigned` or `unverified` and never as verified. A cosign-backed verifier replaces the stub in Phase 3 without touching the domain or usecase code layers.

The chain is content-addressed: VEX references SBOM by SHA-256 checksum, SBOM references the image by digest, and duplicate detection keys on `(image_digest, checksum_sha256)`. Provenance is therefore verifiable from stored evidence alone, independent of signatures.

7.4 Audit logging

Security-sensitive actions write append-only rows to `audit_log` (`actor`, `action`, `resource_type`, `resource_id`, `details_JSONB`, `occurred_at`). The action vocabulary is defined as constants in the domain code layer:

Action	Recorded by
<code>ARTIFACT_ACCEPTED</code> / <code>ARTIFACT_REJECTED</code> / <code>ARTIFACT_WARNING</code> / <code>SIGNATURE_FAILURE</code>	Trust gate, on every upload evaluation (includes trust status, format, warnings, source IP)
<code>TRIAGE_DECISION</code>	Triage usecase, on every human decision
<code>RISK_STATE_TRANSITION</code>	Enrichment, when <code>risk_context.effective_state</code> changes

Action	Recorded by
<code>SBOM_DELETED</code>	<code>DELETE /api/v1/sboms/{id}</code> (soft delete via <code>deleted_at</code> tombstone)

Manual SQL deletes bypass both the audit trail and the immutability guarantees. The operational rule is explicit: prefer `DELETE /api/v1/sboms/{id}` — a soft delete that never removes the underlying rows and always writes an `SBOM_DELETED` audit entry.

7.5 Secrets policy

Secrets are environment-only, never file-committed — verbatim from the README and `themis.yaml.example`: "Secrets (database DSN, SMTP password, API keys) must be set via environment variables — never committed in `themis.yaml`." The concrete set: `THEMIS_DATABASE_DSN` (the only required variable), `THEMIS_SMTP_PASSWORD`, `THEMIS_NVD_API_KEY`, and `THEMIS_WEBHOOK_SECRET`. Any `THEMIS_`-prefixed variable overrides its YAML counterpart, so `themis.yaml` ships with `database.dsn: ""` and remains safe to commit.

7.6 Availability protection and input hardening

- **NVD rate limiting.** The National Vulnerability Database (NVD) client (`internal/adaptor/nvd`) uses a token bucket with capacity 1 (no burst): about 1.5 requests/second with an API key (~45 per 30 s, under NVD's 50/30 s limit) and 0.15 requests/second without (under 5/30 s). These defaults are load-bearing: exceeding them trips NVD's Cloudflare throttle, which returns HTTP 503 challenge pages. `nvd.rate_limit_rps > 0` overrides deliberately.
- **Upload size cap.** `MaxBytes` middleware wraps request bodies with `http.MaxBytesReader` at `upload.max_size_bytes` (default `52428800` — 50 MiB).
- **Envelope validation.** Uploads are a JSON envelope (`format`, `spec_version`, `document`, `artifact_id`, `image_digest`), not raw files; malformed input — including empty-string UUID fields — is rejected with `422` before ingestion.
- **No internal leakage.** Every error response uses the layman envelope `{"error": {code,message,hint}}` from a catalogue of 12 codes; raw PostgreSQL or Go error strings never appear in response bodies (enforced by acceptance criterion AC-23).

7.7 Append-only evidence as a forensic property

The final control is structural. Data-layer L0 tables (`products` through `vex_assertions`) are "append-only, never mutated, never deleted"; raw findings in `component_vulnerabilities` are preserved forever, and VEX application changes only `risk_context.effective_state` — if a VEX statement is revoked, the finding resurfaces rather than having to be re-discovered. Combined with content addressing, the audit log, and soft deletion, this yields a forensic guarantee: for any past risk verdict, the exact SBOM bytes, correlation findings, VEX overlays, and human decisions that produced it remain reconstructable.

8. Quality Engineering

Themis enforces quality through a layered test taxonomy, a script-governed coverage policy, and static architecture gates. The distinguishing property of the regime is that the full suite is a *local* gate executed per task group; Continuous Integration (CI) deliberately runs only a narrow slice. Throughout this chapter, "code layers" refers to the Clean Architecture packages (`domain` / `usecase` / `adapter` / `infrastructure` under `internal/`), not the L0–L3 data layers.

8.1 Test Taxonomy

Tier	Invocation	Scope and mechanism
Unit	<code>make test</code> (<code>go test ./...</code>)	No build tag; runs everywhere.
Integration	<code>make test-integration</code> (<code>go test -tags=integration -p 1 ./...</code>)	~30 files under <code>//go:build integration</code> , mostly stores and <code>internal/infrastructure/</code> { <code>http</code> , <code>db</code> , <code>cli</code> , <code>queue</code> } ; each package's <code>TestMain</code> boots embedded Postgres 16.
Property	<code>make test-property</code> (rapid, default 100 checks as unit tests)	Functions matching <code>Property Prop_</code> ; deep run <code>make test-property RAPID_CHECKS=10000</code> .
Acceptance	<code>go test -tags=integration ./tests/acceptance/</code>	Maps acceptance criteria (AC) to named tests via a meta-test.
End-to-end	<code>scripts/alpine-e2e-gate.sh</code> (manual)	Gates G1–G8 against a running server; not part of CI.

Integration tests and embedded Postgres

Each integration package's `TestMain` starts its own embedded Postgres 16 instance ([github.com/fergusstrange/embedded-postgres v1.34.0](https://github.com/fergusstrange/embedded-postgres)) on a *fixed* port: 15432 for the store package, 15438 for `infrastructure/db`, 15450 for the HTTP helper. Setting `THEMIS_TEST_DATABASE_DSN` substitutes an external Postgres when the embedded binary is unavailable.

`-p 1` is mandatory whenever `-tags=integration` spans multiple packages: parallel test binaries would collide on the fixed ports and data directories. Never drop it from the `make test-integration` recipe.

If embedded Postgres fails to start, the store `TestMain` prints a warning, sets `sharedPostgresFailed`, and the tests *skip* rather than fail — a silently green run. Check stderr for the warning before trusting an integration pass. (`-test.list` mode skips Postgres startup entirely.)

Property-based tests

Property tests use `pggregory.net/rapid` and double as ordinary unit tests. The `test-property` target greps for packages importing `rapid` and passes `-rapid.checks` only to those packages, because the flag is unknown to other test binaries. `Rapid` packages currently include `internal/domain`, the `usecase/{correlation,enrichment,ingestion,triage,watch}` packages, `adapter/{api,notify,parser,trust}`, `infrastructure/queue`, and `tests/acceptance`. Shared generators live in `internal/testutil/gen`: `AnySeverity` and `AnyEffectiveState` deliberately mix random-cased valid values with junk strings so parsers and scorers must tolerate garbage; `Ecosystem / PkgName / PkgVersion` exclude `/` and `@` so Package URL round-trips stay unambiguous. New properties reuse these generators rather than defining ad-hoc ones.

Acceptance tests and the score oracle

`tests/acceptance/criteria_test.go` defines `TestAcceptanceCriteriaCoverage`, a meta-test mapping AC-1..15 (Phase 1) to named test functions; it executes `go test -tags=integration -list .` across all packages and fails if a mapped test name disappears. `criteria_phase2a_test.go` does the same for AC-16..24 (Phase 2a, e.g. `TestAC16_KEVRetroactiveScoreIncrease`); criteria are documented in `docs/current-changes/acceptance-criteria.md`. Renaming an integration test referenced in either mapping requires updating the mapping.

There are two deliberately independent `ComputeRiskScore` implementations — one in `usecase/enrichment`, one in `usecase/triage`. `TestRiskScoreOracleProperty` (`rapid`, untagged) asserts they agree for all severity/state pairs. Any scoring change must be applied to both, or the oracle fails.

Golden finding sets

`internal/testutil/findingset` is the CR-10 regression harness: the canonical finding key is `componentPURL|cveID|source`, `Snapshot` dedupes and sorts, and `AssertGolden` diffs against a committed golden file with an explicit added/removed delta. `UPDATE_GOLDEN=1 go test ...` regenerates goldens (used by `internal/adapter/vexfeed/golden_corpus_test.go` against `testdata/golden/distro_corpus.golden`). Changes to the correlation core are expected to surface here as reviewable drift, never silently. Both `testutil` packages sit intentionally outside the code layers, exempt from depguard and go-cleanarch, and are imported only from `_test.go` files.

8.2 Coverage Governance

`scripts/check-coverage.sh` is the single authority on coverage policy. It runs in two modes: `make coverage-pkg PKG=usecase/enrichment` gates only the touched packages for a task group, while `make coverage` profiles `./internal/{domain,usecase,adapter,infrastructure}/...` plus `tests/acceptance` into `coverage.out` and checks every registered package. Generated `gen/` subdirectories (from `make generate-api`, `oapi-codegen v2.7.1`) are excluded from the computation.

Packages	Threshold
<code>domain</code> , <code>usecase/{ingestion,correlation,triage,vexgen,watch}</code> , <code>adapter/{parser,trust,notify}</code>	100%
<code>adapter/{store,assetgraph,vexfeed}</code> , <code>infrastructure/{db,queue,http,config,metrics,cli}</code>	90%
<code>usecase/enrichment</code> , <code>adapter/osv</code> (overrides)	90%
<code>adapter/{epsskev,exploitdb,redhat}</code> (overrides)	85%
<code>adapter/api</code> (override)	80%

An unregistered package makes the script exit 2 with `unknown package: <pkg>` (register it in `scripts/check-coverage.sh`). New packages are registered first — in the `domain_pkgs` (100%) or `infra_pkgs` (90%) arrays, plus a `threshold_for` case entry for bespoke thresholds — before any code lands.

8.3 Static Gates

Three static gates run alongside the tests. `make lint` runs `golangci-lint`, whose `depguard` configuration enforces the import rules of the code layers (`internal/domain/` imports `stdlib` only; use cases import `domain` only; adapters import `domain` plus `usecase`). `make clean-arch` runs `go-cleanarch` with the explicit mapping `-domain domain -application usecase -interfaces adapter -infrastructure infrastructure` over `./internal`, so the layer rule is enforced twice by independent tools. `make deadcode` runs `golang.org/x/tools/cmd/deadcode -test ./...` with zero tolerance: newly exported but unused code fails the gate.

8.4 Task-Group Gate Procedure

Every task group must pass its task-wise gates — tests, `coverage-pkg` for touched packages, `deadcode`, `integration`, `clean-arch` — followed by a full-codebase `make verify-build` (`clean` + `all`), always in that order. `make check` is the aggregate local gate: `build lint clean-arch coverage deadcode`.

8.5 CI Reality

Only two workflows exist under `.github/workflows/`; CI does *not* run the unit/integration/coverage suite, which is a local gate by design.

- `property-tests.yml` — runs `make test-property` on pull requests (`RAPID_CHECKS=1000`), pushes to `main` (5000), a nightly cron at 03:17 UTC (20000), and `workflow_dispatch` with a custom `rapid_checks` input.
- `changelog.yml` — on tag pushes matching `v*` or `themis-phase-*` (or dispatch), `git-cliff` (`cliff.toml`) regenerates `CHANGELOG.md` and opens a pull request from branch `changelog/update` with explicit `base: main`.

8.6 End-to-End Gate Scripts

`scripts/alpine-e2e-gate.sh` runs `curl/jq/metrics` checks G1–G8 against an already-running server (requires `API_KEY`; `PRODUCT_ID` / `PROJECT_ID` / `SCAN_ID` optional, absent values produce SKIP). G1/G2 `grep` `/metrics` for `themis_epsskev_sync_total` and `themis_vexfeed_sync_total{feed="alpine",status="success"};` G3 checks Vulnerability Exploitability eXchange (VEX) export; G4/G5/G8 assert scan-API enrichment fields (`epss_score` present, `risk_score>0`, `deterministic_level != informational`); G6 checks VEX coverage; G7 checks status CVSS. Any FAIL exits 1. `scripts/run-alpine-e2e-local.sh` bootstraps the full path: build, embedded Postgres on `${THEMIS_E2E_PG_PORT:-15450}`, `make migrate-up`, admin key creation, server on `${THEMIS_E2E_HTTP_PORT:-8080}`, a 120-second feed-sync wait, upload of the canonical fixture `testdata/alpine-busybox.cdx.json` (a minimal CycloneDX 1.6 Software Bill of Materials, SBOM, overridable via `THEMIS_E2E_SBOM`), a 90-second re-enrich wait, then the gate.

`run-alpine-e2e-local.sh` line 64 hardcodes `/opt/homebrew/bin/go` (macOS Homebrew) and breaks on Linux hosts; the Makefile likewise prepends `/opt/homebrew/bin` to `PATH` but falls back to `command -v go`. Patch the script's Go path before running the local e2e bootstrap on Linux.

9. Development Process and Roadmap

Themis is built with a spec-driven process: every behavioral change is first captured as a versioned specification artifact under `openspec/`, implemented against explicit quality gates, then merged back into the canonical spec set. This chapter describes that workflow, the documentation layout that supports it, the release history to date, and the currently active change.

9.1 Spec-Driven Development with OpenSpec

The `openspec/` tree separates the settled behavioral contract from work in flight:

- `openspec/specs/` — 18 canonical, merged capability specifications (one directory per capability, each with a single `spec.md`), covering artifact-registration, artifact-trust, asset-graph, cve-triage, cve-watch, epss-kev, error-ux, exploitdb, intelligence-collector, intelligence-enrichment, notification-service, sbom-ingestion, sbom-management, sbom-parser, sbom-store, system-status, upstream-vex-feeds, and vex-export. These are the behavioral source of truth. They are updated only by syncing or archiving a change — never edited ad hoc.
- `openspec/changes/<name>/` — an active change: `proposal.md`, `design.md`, `tasks.md`, and *delta specs* under `specs/<capability>/spec.md`. Delta specs describe only the difference, under `## ADDED / MODIFIED / REMOVED Requirements` headings, with each `### Requirement:` written as a Request for Comments (RFC)-style normative SHALL statement followed by `#### Scenario:` blocks in `WHEN / THEN` form.
- `openspec/changes/archive/YYYY-MM-DD-<name>/` — completed changes (Phase 1, Phase 2a, v0.2.1, core-model).

The lifecycle is **propose → apply → sync → archive**, driven by five skills in `.claude/skills/` with slash aliases `/opsx:propose`, `/opsx:apply`, `/opsx:explore`, `/opsx:sync`, and `/opsx:archive`. Propose generates all artifacts via the `openspec` command-line interface; apply implements `tasks.md` group by group, with each task group closed by six gates (build, unit tests, coverage, dead code, integration tests, clean-architecture check) and tasks marked complete only when `make check` passes; sync hand-merges delta specs into `openspec/specs/`; archive moves the change directory into the dated archive.

`openspec/STATUS.md` is the authoritative, always-current record of active changes, release tags, and phase state. It is maintained automatically by the propose/apply/archive skills and supersedes any status text found in other documents. It is the first file to read in any session.

`openspec/changes/themis-phase-2/` is an *Architecture Reference*, not an implementable change: it holds the Phase 2 proposal, a design with 16 Architecture Decision Records (ADRs), and `scenario-fresh-deployment.md` (cold-start gap analysis G1–G10). It is referenced by Phases 2a–2c and must never be archived.

9.2 Documentation Organization

Location	Contents
<code>docs/current-changes/PROJECT_CONTEXT.md</code>	Canonical context: data model (data layers L0–L3, distinct from the domain/usecase/adapters/infrastructure code layers), Clean Architecture rules, permanent invariants, quality gates, API conventions.
<code>docs/current-changes/AGENTS.md</code>	Session entry point: context-source priority order, task-group gate procedure, scope guardrail.
<code>docs/current-changes/verification.md</code>	Pre-answer checklist — Correctness C1–C14, Severity S1–S12, Observability O1–O14, Feed Resilience FR1–FR8. Answers touching these areas must cite the rows in a <code>## Verification</code> block; a failing required row blocks completion.
<code>docs/current-changes/acceptance-criteria.md</code>	AC-1..15 (Phase 1) and AC-16..24 (Phase 2a).
<code>docs/current-changes/project-backlog.md</code>	All deferred Phase 2/3 items, decision log, known gaps/defects (D-CVSS-1, D-FEED-1, D-NVD-1, D-LOG-1 — all closed in v0.3.0).
<code>docs/current-changes/NEXT-STAGE.md</code>	v0.4.1+ roadmap: everything deliberately deferred beyond the v0.4.0 thin slice.
<code>docs/architecture/</code>	Architecture book (<code>00-constitution</code> , <code>01-domain</code> , <code>02-backend</code> , <code>diagrams</code>) including ADRs.
<code>docs/release-notes/</code>	Per-release notes, v0.2.0 through v0.3.11.

9.3 Release History

Release	Scope
<code>v0.1.0</code>	Phase 1 — retroactive tag on the Phase 1 commit (<code>a94f3ba</code> , tagged 2026-06-17).
<code>v0.2.0</code> / <code>v0.2.1</code>	Phase 2a plus the Group 31 feed-reliability and Group 16 hardening remainders.
<code>v0.3.0</code>	Core-model and data-layer-L0 refactor CR-1...CR-10 (2026-06-24). Breaking: single squashed migration baseline <code>000001_v030_baseline</code> ; no in-place upgrade from pre-v0.3.0 databases.
<code>v0.3.2</code> – <code>v0.3.11</code>	Maintenance line, no schema migrations (Open Source Vulnerabilities feed fixes, Red Hat VEX scoping, feed registry) plus documentation consolidation; latest tag <code>v0.3.11</code> (2026-07-06).

9.4 Active Change: themis-ai-1 (v0.4.0)

`themis-ai-1` ("Basic AI Enrichment") delivers an advisory Common Vulnerabilities and Exposures (CVE) Summarizer as a deliberately thin slice: one worker, one local model (Ollama `qwen2.5:7b`), and a new, separate `themis-ai` Python service that communicates with the Go backend solely over a

PostgreSQL seam. The `ai` schema is owned by `themis-ai` and migrated with Alembic (not golang-migrate); work discovery is a level-triggered reconcile loop over the `v_ai_enrich_context` view — no queue table (decision D-QUEUE-1); enrichment identity is CVE-grain with an `input_context_hash` (D-GRAIN-1); and the change makes **zero ALTERs** to the data-layer-L0 core-model tables. The eligibility gate admits a CVE when severity is high, it is listed in the Known Exploited Vulnerabilities (KEV) catalog (`kev_listed`), or a public exploit exists (`exploit_public`); the result is `{summary ≤500 chars, primary_weakness CWE-or-null, key_factors ≤5}` (CWE: Common Weakness Enumeration). Planning is complete — proposal, design, spec, and 46 tasks in 9 groups: monorepo restructure, footprint endpoint (D-FOOTPRINT-1), seam contract, backend migration `000002`, transparency API, `ai` schema, harness and reconcile loop, observability and evaluation, then integration and the `v0.4.0` release. Next step: `/opsx:apply`.

9.5 Scope Guardrail and Forward Roadmap

Phase 3 features — rate limiting, cosign signature verification, continuous-integration/continuous-delivery (CI/CD) ingestion, Docker packaging, a web user interface, a Redis queue, and role-based access control (RBAC) — must not be implemented until explicitly requested. Ports for these already exist (`JobQueue`, `SignatureVerifier`) with in-process and stub implementations.

After `v0.4.0`, Phase 2c (`v0.5.0`) plans AI-assisted Vulnerability Exploitability eXchange (VEX) auto-apply; it is blocked on Phase 2b completion and on a knowledge base seeded with real analyst decisions. Until then, decision D-WRITE-1 holds: AI output is advisory-only and never writes `risk_context.effective_state` — state changes require a human.

10. Appendices

These appendices collect the reference material used throughout this specification: the domain vocabulary, the complete environment-variable surface of the server, and the `make` targets that constitute the project's quality gates. Note the two distinct layer vocabularies used in this document: **code layers** refer to the Clean Architecture rings (`internal/domain`, `internal/usecase`, `internal/adapter`, `internal/infrastructure`), while **data layers L0-L3** refer to the storage model (L0 raw immutable inventory, L1 graphs, L2 AI enrichment, L3 human validation, converging into `risk_context`).

10.1 Appendix A: Glossary

Term	Definition
SBOM	Software Bill of Materials — a machine-readable inventory of the components inside an artifact. Themis parses CycloneDX 1.4–1.6, SPDX 2.3–3.0, and Trivy JSON; the uploaded composition is stored in <code>sboms</code> , keyed <code>(artifact_id, sbom_checksum)</code> .
VEX	Vulnerability Exploitability eXchange — machine-readable assertions about whether a product is actually affected by a vulnerability. Stored in <code>vex_documents</code> and <code>vex_assertions</code> ; applied strictly as an overlay (see "VEX overlay").
PURL	Package URL — the canonical component identifier (e.g. <code>pkg:apk/...</code>). With a normalized CVE identifier it forms the correlation and durable-triage key <code>(artifact_id, component_purl, cve_id)</code> . Components without a PURL are skipped during correlation.
CVE	Common Vulnerabilities and Exposures — the public vulnerability identifier scheme. Themis normalizes IDs via <code>domain.NormalizeCVEID</code> (e.g. <code>ALPINE-CVE-*</code> becomes <code>CVE-*</code>).
CVSS	Common Vulnerability Scoring System — the severity metric that supplies the baseline of the composite risk score (CRITICAL 90 / HIGH 70 / MEDIUM 40 / LOW 10).
EPSS	Exploit Prediction Scoring System (FIRST.org) — probability of exploitation. Synced as a gzip CSV feed; contributes up to +30% to the risk score.
KEV	Known Exploited Vulnerabilities — the CISA catalogue of vulnerabilities exploited in the wild. A KEV listing adds a flat +15 to the risk score.
CycloneDX	OWASP-originated SBOM standard; Themis accepts spec versions 1.4, 1.5 and 1.6. The embedded <code>vulnerabilities</code> array is deliberately not ingested.
SPDX	Software Package Data Exchange — the Linux Foundation SBOM standard; versions 2.3–3.0 supported.
Trivy	Aqua Security's scanner; its JSON report format is a third accepted input, normalized by the parser adapter like any SBOM.
OSV	Open Source Vulnerabilities (OSV.dev) — distributed vulnerability database and schema. Used both as a live correlation source at ingest and as downloadable distro feeds (Alpine, Rocky, Wolfi).

Term	Definition
NVD	National Vulnerability Database (NIST) — CVE metadata and CVSS source, consumed through a rate-limited client (token bucket, no burst).
CSAF	Common Security Advisory Framework — OASIS advisory format. Red Hat CSAF VEX is Themis's only overlay-class vendor feed.
RHSA	Red Hat Security Advisory — Red Hat's advisory stream, consumed as a correlation-class source (finding provenance <code>rhsa</code>), distinct from Red Hat's CSAF VEX overlay.
Blast radius	Asset-graph impact multiplier (1.0–2.0×) derived from the number of unique Customers reachable from a component (Component → Microservice → Deployment → Customer), deduplicated per Customer and capped at 10 (<code>intelligence.blast_radius_cap</code>). Defaults to <code>1.0</code> until an asset graph is registered.
VEX overlay	The core invariant: VEX changes only <code>risk_context.effective_state</code> ; rows in <code>component_vulnerabilities</code> are never deleted or modified, so a revoked VEX makes the finding resurface.
Artifact	The L0 registered deployable unit (merged artifacts + images in the v0.3.0 schema), keyed by a globally unique <code>image_digest</code> . It must exist before an SBOM upload passes the trust gate.
Scan report	One correlation run's findings for an artifact (a <code>scan_reports</code> row; N per artifact). "Latest" is defined as <code>ORDER BY scanned_at DESC</code> — there is no <code>is_latest</code> column.
Effective state	<code>risk_context.effective_state</code> — the sole source of truth for a finding's current status: DETECTED, SUPPRESSED, CONFIRMED, IN_TRIAGE, ACCEPTED_RISK, FALSE_POSITIVE, RESOLVED.

10.2 Appendix B: Configuration Reference

Configuration is loaded from an optional `themis.yaml` (read from the working directory, relocatable via `THEMIS_CONFIG_PATH`); environment variables with the `THEMIS_` prefix override every YAML key (`internal/infrastructure/config/load.go`). Defaults below are those shipped in `themis.yaml.example`.

Secrets — the database Data Source Name (DSN), Simple Mail Transfer Protocol (SMTP) password, API keys, webhook secret — must be set via environment variables and never committed in `themis.yaml`. Leave `database.dsn: ""` in the file.

Variable	Purpose	Default
<code>THEMIS_DATABASE_DSN</code>	Required PostgreSQL DSN; missing value fails startup with <code>missing required configuration field: database.dsn</code> . Also required in-shell by <code>make migrate-up / migrate-down</code> .	none

Variable	Purpose	Default
<code>THEMIS_CONFIG_PATH</code>	Path to <code>themis.yaml</code> (read in <code>startup.go</code>).	<code>./themis.yaml</code>
<code>THEMIS_NVD_API_KEY</code>	NVD 2.0 API key; raises the auto rate limit from ≈ 0.15 to ≈ 1.5 requests/s.	unset
<code>THEMIS_NVD_RATE_LIMIT_RPS</code> / <code>THEMIS_NVD_POLL_INTERVAL</code>	NVD client tuning; <code>0</code> means auto, NVD-compliant.	<code>0</code> / <code>6h</code>
<code>THEMIS_SMTP_HOST</code> , <code>PORT</code> , <code>USERNAME</code> , <code>PASSWORD</code> , <code>FROM</code> , <code>USE_TLS</code>	Email notification transport.	port <code>587</code>
<code>THEMIS_TEAMS_WEBHOOK_URL</code>	Microsoft Teams notification channel.	unset
<code>THEMIS_WEBHOOK_SECRET</code>	HMAC-SHA256 secret verifying <code>X-Themis-Signature</code> on <code>/api/v1/webhooks/scan</code> .	unset
<code>THEMIS_TRUST_DEFAULT_POLICY</code>	Trust-gate policy: <code>strict</code> <code>standard</code> <code>permissive</code> .	<code>standard</code>
<code>THEMIS_LOG_LEVEL</code>	<code>debug</code> <code>info</code> <code>warn</code> <code>error</code> ; <code>debug</code> emits per-PURL normalization attempts and Layer 1 rule firings.	<code>info</code>
<code>THEMIS_TEST_DATABASE_DSN</code>	Test-only: points <code>-tags=integration</code> suites at an external PostgreSQL instead of embedded Postgres 16. Not read by the server.	unset (embedded)
<code>THEMIS_EPSSKEV_EPSS_URL</code> , <code>KEV_URL</code> , <code>POLL_INTERVAL</code>	EPSS/KEV feed endpoint and cadence overrides.	<code>24h</code>
<code>THEMIS_EXPLOITDB_CSV_URL</code> , <code>POLL_INTERVAL</code>	Exploit-DB CSV mirror override and cadence.	<code>24h</code>
<code>THEMIS_VEXFEED_RHEL_VEX_URL</code> , <code>RHEL_CSAF_URL</code> , <code>ALPINE_OSV_URL</code> , <code>ROCKY_OSV_URL</code> , <code>WOLFI_OSV_URL</code> , <code>POLL_INTERVAL</code>	Vendor feed URL overrides for the built-in feeds; <code>THEMIS_VEXFEED_RHEL_URL</code> is a deprecated alias kept one release.	<code>24h</code>
<code>THEMIS_INTELLIGENCE_BLAZT_RADIUS_CAP</code>	Unique-Customer cap for the blast-radius multiplier.	<code>10</code>
<code>THEMIS_GITHUB_TOKEN</code>	Config-wired but currently unused — the GitHub Security Advisory (GHSA) adapter is not implemented.	unset

Additional tunables follow the same pattern: `THEMIS_SERVER_PORT` (default `8080`) and server timeouts, `THEMIS_DATABASE_MAX_POOL_SIZE` (`20`), `THEMIS_WORKER_POOL_SIZE` (`10`) / `MAX_RETRY` (`3`) /

`_BASE_DELAY`, `THEMIS_UPLOAD_MAX_SIZE_BYTES` (52428800, 50 MiB) / `_TIMEOUT` (5m), and `THEMIS_OSV_RATE_LIMIT_RPS` / `_POLL_INTERVAL`.

10.3 Appendix C: Build and Verification Command Reference

All targets live in the repository `Makefile`; the per-package coverage floors are enforced by `scripts/check-coverage.sh`.

Target	Purpose
<code>make build</code>	Compiles the single binary to <code>./bin/themis</code> (the default target).
<code>make verify-build</code>	<code>make clean</code> then <code>make all</code> — the mandatory full-codebase rebuild after every task group.
<code>make test</code>	Unit tests: <code>go test ./...</code> (no build tag).
<code>make test-integration</code>	<code>go test -tags=integration -p 1 ./...</code> — boots embedded Postgres 16 on fixed ports (store 15432, db 15438, ...); <code>-p 1</code> is mandatory to avoid port collisions. Set <code>THEMIS_TEST_DATABASE_DSN</code> to use an external database.
<code>make test-property</code>	Property-based tests (pgregory.net/rapid); default 1000 examples, deep runs via <code>RAPID_CHECKS=20000</code> . The flag is passed only to packages that import rapid.
<code>make lint</code>	Runs golangci-lint (v2 config), including the <code>depguard</code> import rules.
<code>make clean-arch</code>	go-cleanarch check of the code-layer dependency rule over <code>./internal</code> ; fails on any inward-pointing violation.
<code>make deadcode</code>	golang.org/x/tools/cmd/deadcode -test <code>./...</code> — zero tolerance; every exported symbol needs a consumer.
<code>make coverage</code>	Full-repo coverage to <code>coverage.txt</code> , then per-package floor enforcement (100% for domain and most usecase/parser/trust/notify packages; 90/85/80% tiers elsewhere; <code>gen/</code> excluded).
<code>make coverage-pkg</code> <code>PKG=<path></code>	Task-group-scoped coverage; <code>PKG</code> is the path under <code>internal/</code> without the prefix, e.g. <code>PKG=usecase/enrichment</code> (space-separated lists allowed). Unregistered packages exit 2.
<code>make check</code>	Composite gate: build + lint + clean-arch + coverage + deadcode. Required before every commit.
<code>make migrate-up</code> / <code>make migrate-down</code>	Manual golang-migrate runs over <code>migrations/</code> , invoked with <code>-tags postgres</code> (the driver registers only then); both require <code>THEMIS_DATABASE_DSN</code> exported in the same shell. The server also auto-migrates at boot.
<code>make generate-api</code>	Regenerates OpenAPI server stubs from <code>api/openapi.yaml</code> via oapi-codegen v2.7.1 into <code>internal/adapters/api/gen/api.gen.go</code> (checked in, coverage-exempt).

Gate order per task group is fixed: package unit tests, `coverage-pkg` , `deadcode` , package integration tests, `clean-arch` — then, always last, `make verify-build` over the whole repository.

After a code change, rebuild the binary before resetting the database — running an old binary against a fresh database re-creates stale data. There is no in-place upgrade from a pre-`v0.3.0` schema; drop and recreate, then re-run `make migrate-up`.